

# AES (Advanced Encryption Standard)

## → Características fundamentais

A criptografia converte os dados em uma forma inteligível chamada texto cifrado; descifrar o texto cifrado converte os dados de volta em sua forma original, chamada texto simples.

• O algoritmo da AES é capaz de usar chaves criptográficas de 128, 192 e 256 bits para criptografar e descifrar dados em blocos de 128 bits.

• AES é uma variante da família Rijndael de algoritmos de criptografia de bloco simétricos.

• O AES pode ser usado em diversos contextos, como: segurança sem fio, segurança de processador, criptografia de arquivos e SSL/TLS.

• O padrão antigo (DES) só conseguia até 56 bits.

• As combinações possíveis para criptografar AES de 256 bits são  $2^{256}$ .

## → Como funciona?

• Os dados são empilhados em uma matriz  $4 \times 4$  de 128 bits (16 bytes) que é conhecida como "estado".

|                                                                                                    |    |    |     |     |
|----------------------------------------------------------------------------------------------------|----|----|-----|-----|
| <div>             1 byte<br/>             8 bits = 1 palavra<br/>             Hex           </div> | A0 | A4 | A8  | A12 |
|                                                                                                    | A1 | A5 | A9  | A13 |
|                                                                                                    | A2 | A6 | A10 | A14 |
|                                                                                                    | A3 | A7 | A11 | A15 |

16 bytes = 128 bits

|         | tamanho da chave (UK us) | term. blocos (word) | nº de Rodadas |
|---------|--------------------------|---------------------|---------------|
| AES-128 | 4                        | 4                   | 10            |
| AES-192 | 6                        | 4                   | 12            |
| AES-256 | 8                        | 4                   | 14            |

### • Operações Criptográficas:

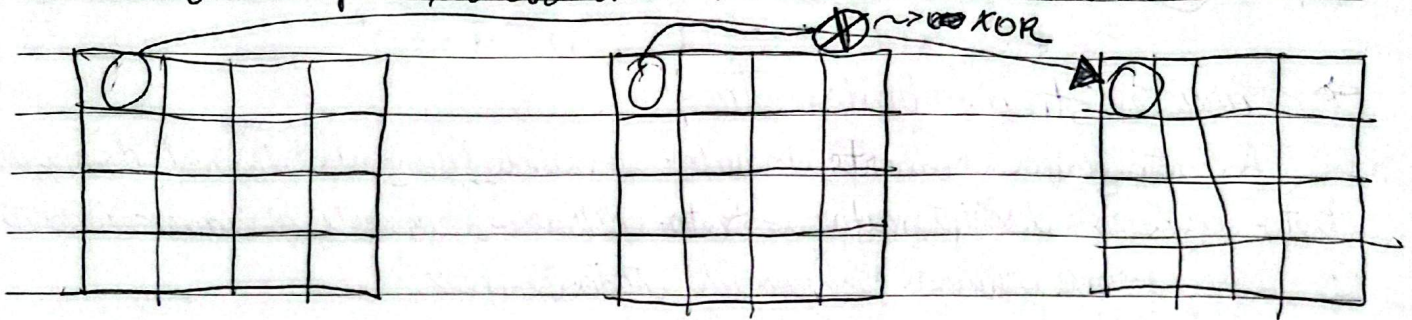
#### 1. Expansão de chave e Add Round Key

• O tamanho da chave determina o número de Rodadas de embaralhamento que serão realizadas.

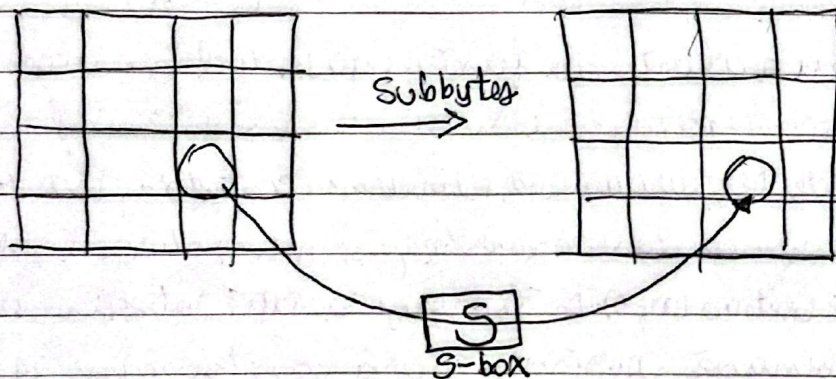
$N_b (N_r + 1)$ , quantidades de chaves

FORONI

• A operação Add Roundkey obtém o estado atual dos dados e executa a operação booleana XOR na subchave round atual.



## 2. Sub Bytes



• Literalmente o bloco de 128 bits (16 bytes) é substituído um por um por outros números de uma tabela com números pseudo-aleatórios.

## 3-Shift Rows

• A primeira linha permanece inalterada, a segunda desloca os bytes em uma posição com a quebra de linha. A terceira linha desloca um a mais, totalizando duas posições com quebra de linha. Da mesma forma, isso significa que a quarta linha desloca os bytes para a esquerda em um total de 3 posições com quebra de linha.

| -X | $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ |              | $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ |
|----|----------|----------|----------|----------|--------------|----------|----------|----------|----------|
| ←  | $a_{10}$ | $a_{11}$ | $a_{12}$ | $a_{13}$ | Shift Rows → | $a_{11}$ | $a_{12}$ | $a_{13}$ | $a_{10}$ |
| ←  | $a_{20}$ | $a_{21}$ | $a_{22}$ | $a_{23}$ |              | $a_{22}$ | $a_{23}$ | $a_{20}$ | $a_{21}$ |
| ←  | $a_{30}$ | $a_{31}$ | $a_{32}$ | $a_{33}$ |              | $a_{33}$ | $a_{30}$ | $a_{31}$ | $a_{32}$ |

#### 4. MixColumns

• Cada coluna de 4 bytes é tratada como um vetor coluna e multiplicada por uma matriz fixa de constantes sob o corpo de Galois  $GF(2^8)$ :

$$\begin{bmatrix} S'_0 \\ S'_1 \\ S'_2 \\ S'_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} S_0 \\ S_1 \\ S_2 \\ S_3 \end{bmatrix}$$

• Para calcular o primeiro byte da nova coluna ( $S'_0$ ), a equação é:

$$S'_0 = (\{02\} \cdot S_0) \oplus (\{03\} \cdot S_1) \oplus (\{01\} \cdot S_2) \oplus (\{01\} \cdot S_3)$$

• Matemática Subjacente: Corpo de Galois  $GF(2^8)$

- As operações aritméticas no AES não utilizam a matemática comum de inteiros, mas sim a aritmética de polinômios no corpo finito  $GF(2^8)$

• **Adição:** No  $GF(2^8)$ , a adição é equivalente à operação lógica XOR bit a bit. Não existe "vai um" (less carry additions)

• **Multiplicação por  $\{01\}$ :** Multiplicar qualquer byte por  $\{01\}$  resulta no próprio byte

$$\{01\} \cdot b = b$$

• **Multiplicação por  $\{02\}$ :** multiplicar por  $\{02\}$  equivale a multiplicar por  $x$  no domínio de polinômios. Em nível de bits:

1- Desloca-se o byte 1 bit para a esquerda ( $b \ll 1$ )

2- Tratamento de estouro (overflow): Se o bit mais significativo (MSB/bit 7) era 1 antes do deslocamento, ocorre um estouro no grau  $x^8$ . Para manter o resultado dentro do  $GF(2^8)$  aplica-se um XOR com a constante  $1B_{16}$  ( $00011011_2$ ), que representa o polinômio irreduzível:

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

• Multiplicação por {03}: Como  $\{03\} = \{02\} \oplus \{01\}$ , a multiplicação por  $\{03\}$  deriva da soma das duas operações anteriores:  
$$\{03\} \cdot b = (\{02\} \cdot b) \oplus b = \text{Xtime}(b) \oplus b$$

### → Ordem das Operações

A ordem em qual essas operações são executadas são:

- 1- Expansão de chave define a lista de chave de rodadura que é usada em cada rodada mais uma rodada adicional (e inicial, como você verá),
- 2- Add Round é a primeira etapa para ofuscar os dados. Imediatamente, temos dados embaralhados.
- 3- Agora entramos mais rodadas de intensa mistura de dados. Novamente, dependendo do bit de cifra selecionado, o número de rodadas será diferente.
- 4- Para 9, 11 ou 13 rodadas, dependendo da seleção do bit de cifra, a seguinte será executada nesta ordem:
  - 5- Sub Bytes
  - 6- Shift Rows
  - 7- Mix Columns
  - 8- Add Round
  - 9- No 10<sup>o</sup>, 12<sup>o</sup> ou 14<sup>o</sup> rodada, respectivamente, que executam a seguinte final das operações, que são as seguintes:
    - 10- Sub Bytes
    - 11- Shift Rows
    - 12- Add Round

# Protocolo SPI

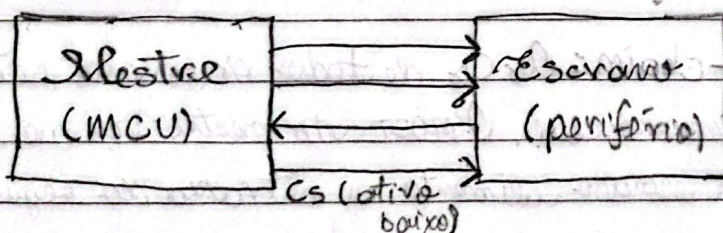
## → Visão Geral do SPI

- É uma interface usada em MCUs e ICs, na qual a afinidade com sensores, ADCs, DACs, Registradores de deslocamento, SRAM, etc.). Interface síncrona, full-duplex e mestre-escravo.
- Mestre ou escravo são sincronizados em borda de subida ou descida, e ambos podem transmitir ao mesmo tempo.

## → Sinais e Conexões

- O dispositivo que gera o CLK é o mestre, cho apenas um mestre, com um ou vários escravos.

| Sinal         | Função                                                                                                                           |
|---------------|----------------------------------------------------------------------------------------------------------------------------------|
| SCLK (SPICLK) | CLK gerado pelo mestre; sincroniza a transferência.                                                                              |
| CS            | Chip select, normalmente ativo em nível baixo; o mestre coloca 0 para selecionar o escravo e 1 para desconectá-lo do barramento. |
| MOSI          | Master Out, Slave In: dados do mestre p/ o escravo                                                                               |
| MTSO          | Master in, Slave Out: dados do escravo p/ o mestre                                                                               |



## → Transmissão de dados

- Para iniciar a comunicação, o mestre gera o CLK e coloca CS em nível 0. Durante a transferência, os bits são deslocados (enviados) e amostrados (lidos) ao mesmo tempo: a borda de CLK sincroniza as duas ações.

## → Polaridade e fase do clock (CPOL e CPHA)

• O mestre deve escolher a polaridade e a fase de acordo com o escravo:

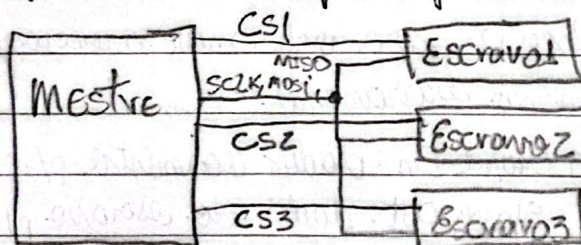
- **CPOL**: nível do clock em repouso (idle): 0 = baixo, 1 = alto

- **CPHA**: define em qual borda do clock os dados são amostrados e em qual são deslocados.

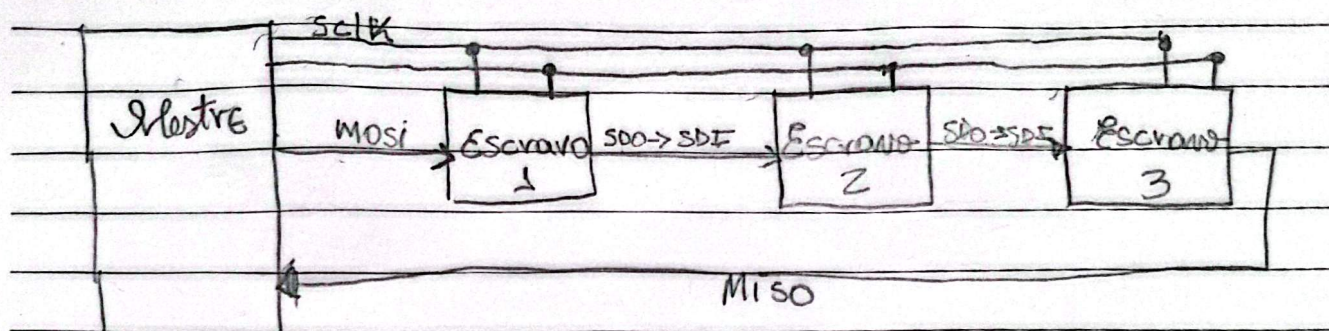
## → Configuração com vários escravos

• Vários escravos podem compartilhar um único mestre, em dois modos:

- **Modo Regular**: Cada escravo precisa de um CS individual, e SCK, MOSI e MISO são compartilhados. Se dois CS ficarem ativos ao mesmo tempo, os dados em MISO são corrompidos, pois o mestre não sabe qual escravo está transmitindo. A desvantagem é que o número de linhas de CS cresce com o número de escravos, consumindo GPIOs do mestre (uma solução é usar um mux para gerar os CS).



- **Modo daisy-chain**: Os CS de todos os escravos são ligados juntos e todos recebem o mesmo clock. O MOSI do mestre vai ao primeiro escravo, a saída de cada escravo alimenta a entrada do seguinte e a saída do último volta ao MISO do mestre. Assim, o mestre usa apenas 4 GPIOs, qualquer que seja o número de escravos. Em contrapartida, o número de pulsos de clock cresce com a posição do escravo: em um sistema de 8 bits são necessários 24 pulsos para o dado chegar ao 3º escravo, contra apenas 8 no modo regular. Nem todo dispositivo suporta daisy-chain.



### → Estudo de Casos: Chaves e Muxes com interfaces SPI

• Em sistemas com muitas chaves analógicas, controlar cada chave por GPIO consome muitos pinos.

• U0810 tem a chave quadrupla ADG 1412 que exige 4 GPIOs, assim serão necessários com 4 ADG 1412  $4 \times 4 = 16$  GPIOs. As alternativas para poder atenuar isso seria:

- Conversor serial-paralelo: Configurado por SPI e com saídas paralelas para as chaves; reduz GPIOs, mas aumenta a lista de componentes.

- Chaves com interface SPI: dispensam o conversor; com as individuais U0810 tem 7 GPIOs para as quatro chaves (SCLK, MOSI, MISO e UCS)

- SPI em daisy-chain: apenas 4 GPIOs, independente do número de chaves.